

Architecture & Production Design

GraphOne / FrontierAtlas Intelligence Graph — ingestion pipeline.

0. What is built (trial scope)

Vertical	Source (implemented)	Extraction
Startups	Y Combinator directory via its Algolia index (public search key scraped live)	direct field map — no LLM , zero hallucination surface
Products	Product Hunt v2 GraphQL (AI topics); TheresAnAIForThat via ScraperAPI as the anti-bot demo + top-up	LLM draft → guarded (startupName kept only if present in source or a known canonical)
Research papers	arXiv Atom API → GitHub repo discovery (abstract, comment, full-text HTML) → live star count via GitHub REST	structured; LLM optional. require_github keeps only rows that carry metrics
News	5 RSS feeds (VentureBeat, TechCrunch, The Verge, MIT Tech Review, Ars Technica)	full-text extraction; 24 h hard gate
Jobs	Jobicy, Remoteive, RemoteOK, We Work Remotely, Himalayas	company/date/remote from the feed; role_family = deterministic classifier; LLM only fills a missing company

LLM chain (config llm.chain): Groq openai/gpt-oss-120b → Gemini gemini-flash-latest (900k context, called over REST) → DeepSeek deepseek/deepseek-chat via OpenRouter. A content-keyed on-disk cache (data/cache/extraction_cache.sqlite) makes a rate-limited run resumable for free.

Anti-bot (Phase V), demonstrated not just documented: direct fetch → ScraperAPI (residential IPs, render=false) → ScraperAPI render=true (server-side browser) → local Playwright. TheresAnAIForThat returns HTTP 403 to a direct request on every page; the pipeline pulls it successfully through this ladder.

Output: append-only JSONL (source of truth) + a 6-tab .xlsx workbook (**Startups · Products · Research Papers · Jobs · News · Entity Mapping Log**); a Google Sheets API writer is included for environments that permit service-account keys.

1. Scale strategy — collecting 500,000+ records without manual intervention

Shape of the work. Acquisition is I/O-bound (thousands of HTTP round-trips); extraction is rate-limited by LLM providers; both are embarrassingly parallel per record. So the system is a **queue-driven fan-out of stateless workers**, scaled horizontally.

```
Seed/Frontier [ url_queue ] [ Crawl workers [ extract_queue ] [ Extract workers [ resolve+load ] (per source) (Redis/SQS) (async, N pods) (Redis/SQS) (LLM chain, M pods) (DB + Sheets/API)
```

- **Frontier generation.** Each source declares how to enumerate itself in sources.yaml (API pagination, sitemap, list-page pattern). A scheduler expands sources into URL ranges and fills url_queue. Adding a source is a YAML edit — no code change (the trial code already works this way).
- **Crawl workers** are the current AsyncFetcher loop, unchanged, one bounded event loop per pod. Throughput scales by pod count. Politeness is enforced per-host by a shared token bucket in Redis so N pods still respect one site's rate limit collectively.
- **Extract workers** run the LLMorchestrator fallback chain. They scale independently of crawl workers because their bottleneck is provider quota, not bandwidth.
- **Backpressure.** Bounded queues; when extract_queue is full, crawl workers block — the system self-throttles to the slowest stage instead of building an unbounded backlog.
- **Idempotency & resumability.** Every stage keys off the normalized-URL hash and is safe to retry. A killed pod loses at most its in-flight messages, which return to the queue on visibility timeout. A full re-run re-processes nothing (see §3).

- **Cost/latency controls.** Structured verticals (arXiv) bypass the LLM entirely. Cheap tier first. Aggressive HTML caching so re-crawls are nearly free. Batch small docs into one LLM call where schema allows.
- **Trial vs. prod.** The trial runs the same code with the in-process `asyncio.Queue` and a SQLite `SeenStore`; production swaps `seen_store.backend: redis` and points the queue at SQS. No business-logic change — that is the design constraint the brief asks for.

2. Handling 413 (context overflow) and 429 (rate limits) at scale

429 — rate limits.

- Per-provider **client-side limiter** (token bucket) sized to the plan's documented RPM/TPM, shared across extract pods via Redis so aggregate load stays under quota.
- On a 429 response: honour `Retry-After` if present, else **exponential backoff with full jitter** (`base · 2^attempt · rand(0.5, 1.5)`), capped. `attempts_per_tier` retries, then **fall through to the next tier** in the chain (Groq → Gemini Flash → DeepSeek). Different providers = independent quota pools, so a fallback is also a load-shedding valve.
- The content-keyed extraction cache means a run interrupted by a rate-limit storm resumes without re-paying for anything already extracted.
- Circuit breaker: if a tier returns 429 for >X% of calls over a window, skip it for a cooldown period instead of hammering it.
- The same policy applies to source sites (HTTP 429/503) in `AsyncFetcher`.

413 — context window / payload too large.

- **Pre-flight guard.** `ChunkPlan.budget_chars() = max_input_tokens · safety_ratio` minus reserves for prompt + response. Cheap char/token estimate, deliberately conservative — most docs never approach the limit.
- **Structural chunking.** Oversized docs split on headings → paragraphs → sentences (never mid-word), into **overlapping** windows so an entity that straddles a boundary still appears whole somewhere.
- **Information-density selection.** For single-entity extraction we send the highest-signal chunk (`pick_primary_chunk`) rather than map-reducing every chunk — cheaper and less error-prone. Map-reduce is reserved for multi-entity pages.
- **Runtime feedback loop.** An actual 413 from the provider triggers `ChunkPlan.shrink()` (halve the safety ratio), re-chunk, retry — the budget adapts instead of being a fixed guess.
- Each tier has its own `max_input_tokens`, so falling back can also mean "smaller context" and the doc is re-chunked for the new tier automatically.

3. Freshness tracking — never processing the same article/job twice across nodes

- **Canonical key.** `sha1(normalized_url)` where normalization lowercases scheme/host, strips the fragment, sorts query params, and drops tracking params (`utm_*`, `fbclid`, ...). Same article via 3 URLs → one key.
- **Shared dedupe set.** `SeenStore` — SQLite locally, **Redis SET** in production (or a **Bloom filter** when the keyspace makes exact sets expensive; false positives only skip, never duplicate). `add_if_new()` is a single atomic `SADD-and-check`, so two crawler nodes racing on the same URL, exactly one wins.
- **24h window is the hard gate** for news/jobs: parse the publication date (`dates.parse_date` handles ISO, RFC-822, "2 hours ago", "yesterday", localized formats, missing meta tags) and require it within `window_hours`.
- **Heuristic fallback** when a source has no reliable date: `FreshnessState` keeps a per-source high-water mark (timestamp of the newest item seen on the last successful run); an undated item counts as new only if it sorts after that mark, or the source has never been run.
- **Content-hash backstop** (prod): hash of normalized title+body catches the same story re-published at a new URL.
- Dedupe state is external and shared, so horizontal scaling doesn't reintroduce duplicates; a crash mid-run is safe because keys are written as items are accepted, and the whole thing is replay-safe.

4. Storage strategy

Concern	Choice	Why (short)
---------	--------	-------------

Primary store	PostgreSQL (managed)	Structured, schema-versioned records; transactions for resolve+load; JSONB for content . data; date/freshness indexing; mature ops. One store, no premature exotica.
Dedupe / limits / queues	Redis	O(1) seen-set membership, shared token buckets, light queues. Ephemeral, horizontally reachable.
Relationship graph	Neo4j (Postgres edge tables until it hurts)	It's an <i>Intelligence Graph</i> — startup ■ product ■ paper ■ author ■ job. Multi-hop queries are native there; Postgres stays system of record, the graph is a rebuilt projection.
Vector store	pgvector, then Qdrant/Pinecone at scale	Embeddings drive semantic dedupe, entity resolution beyond string similarity, and graph similarity search. Stay in Postgres until recall/latency forces a split.
Raw HTML	Object storage, keyed by URL hash	Immutable audit trail (every record traces to its bytes — the anti-hallucination guarantee); re-extract without re-crawling on schema/model change.
Run outputs	JSONL + 6-tab .xlsx (Sheets API writer where SA keys are allowed)	JSONL = portable source of truth + diff target; workbook = human deliverable, rewritten per run.

Data-integrity guarantee. `source.url` required and validated on every record; raw HTML retained; LLM instructed to emit `null` not guess; validation failures on all tiers drop the record with a log line. No fabricated rows reach the store.

5. Anti-bot & scale thinking (Phase V summary)

Cheap path first (`aihttp`, rotating UA, per-host politeness). Escalate a domain **only** on a challenge signal (403/503 + short body, Cloudflare / DataDome markers): ScraperAPI residential IPs → ScraperAPI server-side render → headless Chromium with realistic UA/viewport/locale/TZ, `navigator.webdriver` patched, human-ish pacing, one context per proxy identity, proxy rotation on challenge, exponential backoff — and **never solve CAPTCHAs**: park the domain and alert. Aggressive HTML caching minimises hits. At 500k scale this is a separate pool of render/proxy workers on the same `extract_queue`, so protected sources never slow the main crawl.